Solidity合约升级详解与实战

目录

1. 合约升级基础概念
2. 透明代理升级(Transparent Proxy)
3. UUPS升级模式
4. 两种升级模式对比
5. 完整Demo实战
6. 部署和测试指南
7. 最佳实践和安全考虑

---

1. 合约升级基础概念

1.1 为什么需要合约升级

在传统的区块链合约开发中，智能合约一旦部署到区块链上就是不可变的。这种不可变性虽然提供了去中心化和信任的基础，但也带来了一些实际问题：

业务需求变化：随着项目的发展，可能需要添加新功能或修改现有逻辑
漏洞修复：发现安全漏洞时需要及时修复
性能优化：随着技术发展，可能需要优化合约性能
法规遵循：监管要求变化时需要调整合约逻辑

1.2 代理模式的核心原理

合约升级的核心思想是使用**代理模式（Proxy Pattern）**，将合约的存储和逻辑分离：

```
用户交互 → 代理合约(Proxy) → 逻辑合约(Implementation)
              ↓
          持久化存储
```

关键概念：

- **代理合约(Proxy)**：负责存储数据和转发调用，地址永远不变
- **逻辑合约(Implementation)**：包含业务逻辑，可以被升级替换
- **delegatecall**：核心机制，在代理合约的存储上下文中执行逻辑合约的代码

1.3 delegatecall机制详解

`delegatecall`是Solidity中的一个特殊调用机制：

```
// 普通call：在目标合约的上下文中执行
target.call(data); // msg.sender = 调用者, storage = 目标合约的storage

// delegatecall：在当前合约的上下文中执行目标合约的代码
target.delegatecall(data); // msg.sender = 原始调用者, storage = 当前合约的storage
```

这使得我们可以在代理合约中执行逻辑合约的代码，但所有的状态变化都保存在代理合约中。

---

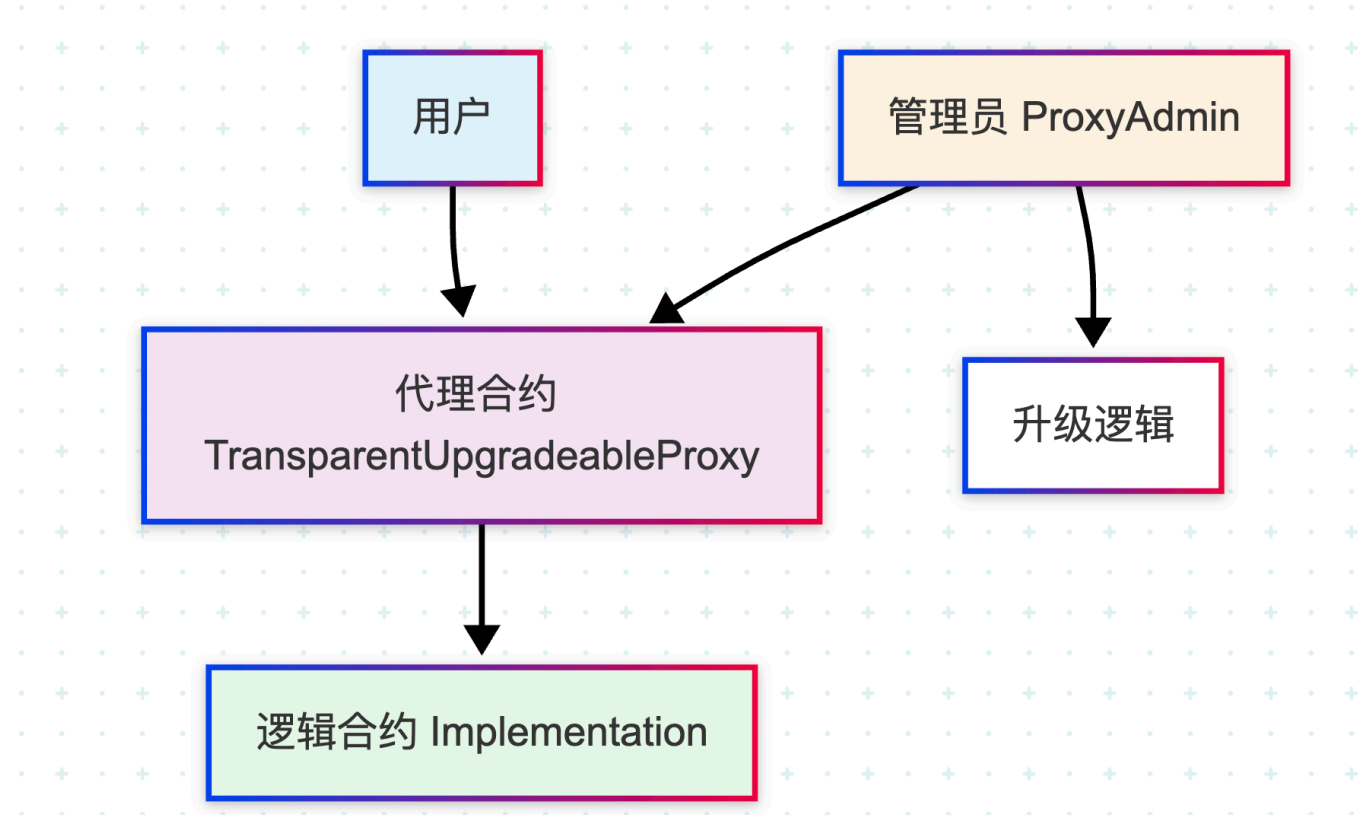
2. 透明代理升级(Transparent Proxy)

2.1 透明代理的设计理念

透明代理升级是OpenZeppelin提出的一种升级模式，其核心理念是**完全透明性**：

- 对于用户来说，他们只需要知道代理合约的地址
- 用户调用的所有函数都会被透明地转发到逻辑合约
- 升级过程对用户完全透明

2.2 透明代理的架构组件



核心组件：

1. **TransparentUpgradeableProxy**：代理合约，负责转发调用和存储数据
2. **ProxyAdmin**：管理合约，只有它可以升级代理合约
3. **Implementation Contract**：逻辑合约，包含实际的业务代码

2.3 函数选择器冲突解决

透明代理最重要的特性是解决了**函数选择器冲突**问题：

```
// 如果代理合约和逻辑合约都有相同的函数签名，会发生冲突
// 透明代理通过权限隔离解决这个问题：

if (msg.sender == admin) {
    // 管理员调用：执行代理合约的管理函数
    // 如：upgradeTo(), changeAdmin()
} else {
    // 普通用户调用：转发到逻辑合约
    // delegatecall到implementation合约
}
```

2.4 透明代理的工作流程

```
// 透明代理的核心逻辑伪代码
fallback() external payable {
    if (msg.sender == _getAdmin()) {
        // 管理员调用管理函数
        if (sig == upgrade.selector) {
            _upgrade(newImplementation);
        } else if (sig == changeAdmin.selector) {
            _changeAdmin(newAdmin);
        }
        // ... 其他管理函数
    } else {
        // 普通用户调用，转发到逻辑合约
        address impl = _getImplementation();
        assembly {
            calldatacopy(0, 0, calldatasize())
            let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)
            returndatacopy(0, 0, returndatasize())
            switch result
            case 0 { revert(0, returndatasize()) }
            default { return(0, returndatasize()) }
        }
    }
}
```

---

3. UUPS升级模式

3.1 UUPS的设计哲学

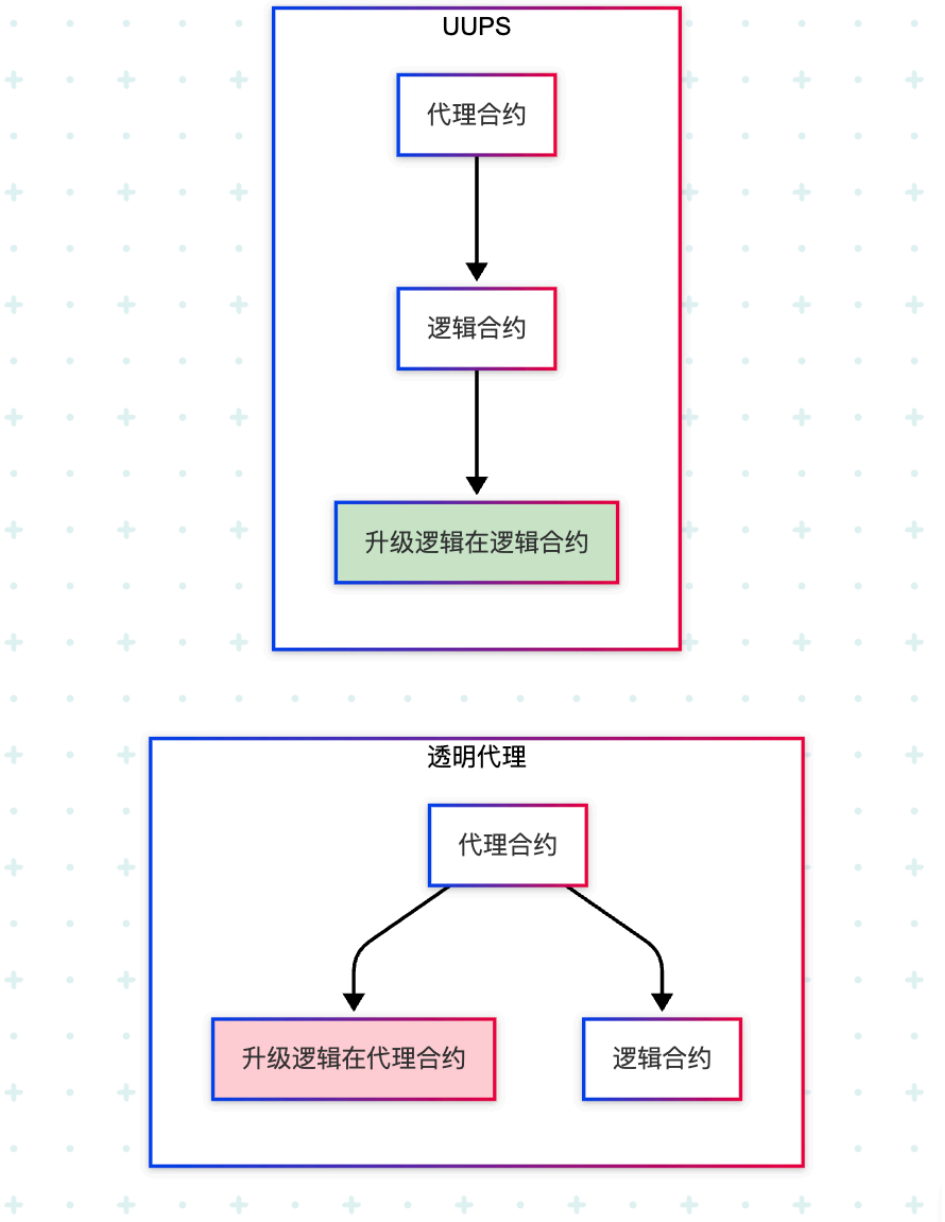
UUPS（Universal Upgradeable Proxy Standard）是一种更加Gas高效的升级模式，其核心理念是**将升级逻辑放在逻辑合约中**而不是代理合约中。

UUPS名称由来：

- **U**niversal：通用的

- **U**pgradeable：可升级的
- **P**roxy：代理
- **S**tandard：标准

3.2 UUPS与透明代理的核心区别



关键差异：

特性	透明代理	UUPS
升级逻辑位置	代理合约中	逻辑合约中
Gas成本	较高（每次调用都检查权限）	较低（升级时才检查）
代理合约复杂度	高	低
升级安全性	代理合约保证	逻辑合约保证

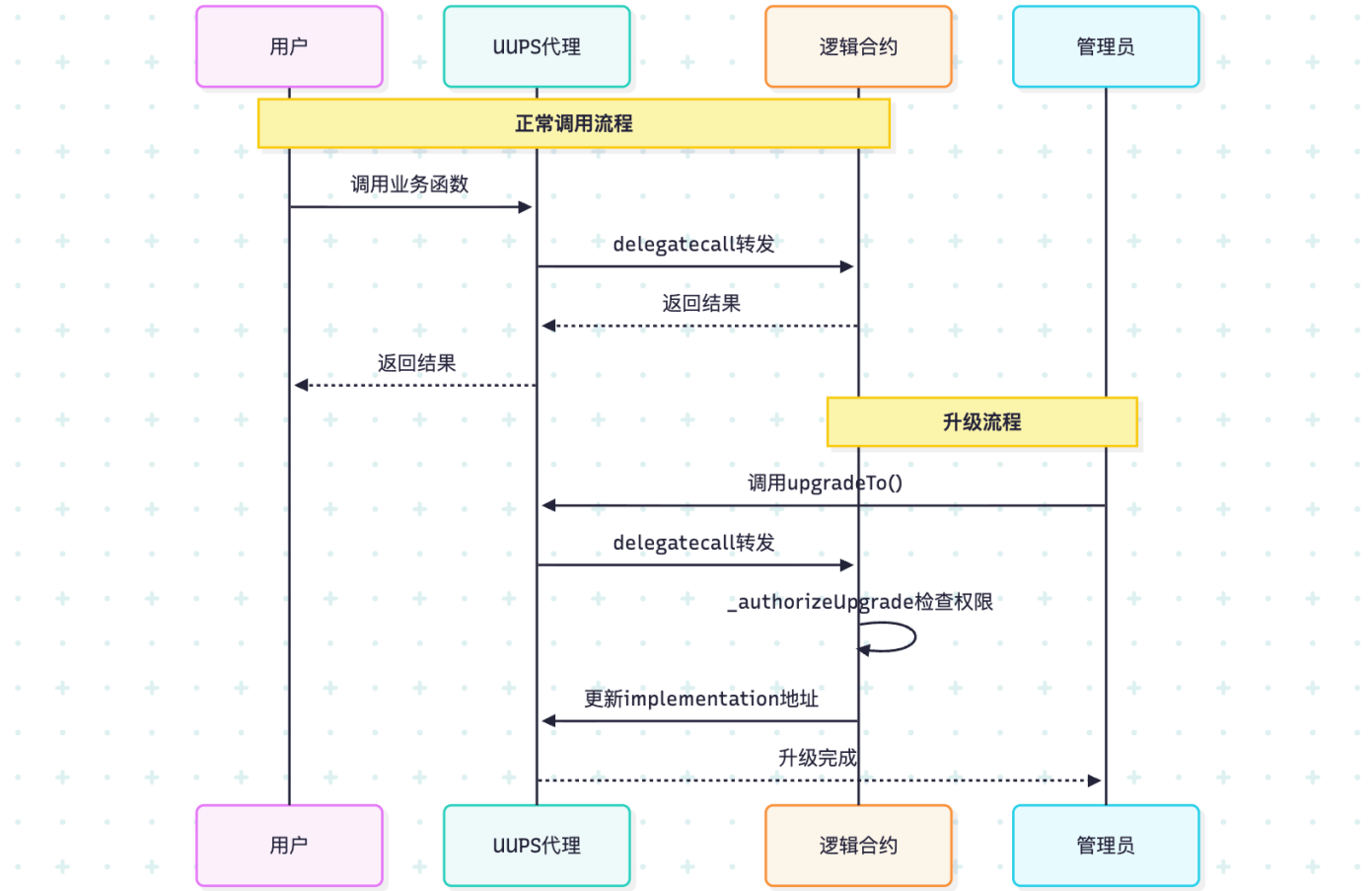
3.3 UUPSUpgradeable的核心机制

```
// UUPS升级的核心逻辑
abstract contract UUPSUpgradeable {
    // 升级函数，必须在逻辑合约中实现权限控制
    function upgradeTo(address newImplementation) external virtual onlyProxy {
        _authorizeUpgrade(newImplementation);
        _upgradeToAndCallUUPS(newImplementation, new bytes(0), false);
    }

    // 必须在继承合约中实现，定义谁可以升级
    function _authorizeUpgrade(address newImplementation) internal virtual;

    // 验证调用来自代理合约
    modifier onlyProxy {
        require(
            address(this) != __self,
            "Function must be called through delegatecall"
        );
        _;
    }
}
```

3.4 UUPS的工作原理



4. 两种升级模式对比

4.1 详细对比分析

对比维度	透明代理	UUPS	推荐场景
Gas消耗	每次调用+2000 gas	每次调用+200 gas	UUPS更优
安全性	升级逻辑隔离在代理合约	依赖逻辑合约实现	透明代理更安全
开发复杂度	低（框架自动处理）	中（需要实现权限控制）	透明代理更简单
升级灵活性	标准化，不易出错	灵活，但需谨慎	根据需求选择
存储冲突风险	框架自动处理	需要手动管理	透明代理更安全

4.2 选择建议

选择透明代理的场景：

- 对安全性要求极高的项目
- 团队对升级模式不够熟悉
- 需要频繁升级的合约
- 资金量较大的DeFi协议

选择UUPS的场景：

- 对Gas成本敏感的高频应用
- 团队有丰富的合约升级经验
- 需要自定义升级逻辑的场景
- 追求极致性能的应用

---

5. 完整Demo实战

5.1 项目结构设置

首先创建项目结构：

```
upgradeable-contracts-demo/
├── contracts/
│   ├── transparent/
│   │   ├── BoxV1.sol
│   │   └── BoxV2.sol
│   └── uups/
│       ├── BoxUUPSV1.sol
│       └── BoxUUPSV2.sol
├── scripts/
│   ├── deploy-transparent.js
│   ├── deploy-uups.js
│   ├── upgrade-transparent.js
```

```
│   └── upgrade-uups.js
├── test/
│   ├── transparent-test.js
│   └── uups-test.js
├── hardhat.config.js
└── package.json
```

5.2 透明代理Demo实现

5.2.1 BoxV1.sol - 初始版本

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";

/**
 * @title BoxV1
 * @dev 透明代理升级演示合约 - 第一版
 * 提供基础的数值存储和检索功能
 */
contract BoxV1 is Initializable, OwnableUpgradeable {
    // 存储的数值
    uint256 private _value;

    // 事件定义
    event ValueChanged(uint256 newValue);

    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() {
        _disableInitializers();
    }

    /**
     * @dev 初始化函数，替代构造函数
     */
    function initialize() public initializer {
        __Ownable_init();
        _value = 0;
    }

    /**
     * @dev 存储数值
     * @param newValue 要存储的新数值
     */
    function store(uint256 newValue) public {
        _value = newValue;
        emit ValueChanged(newValue);
    }
```

```
    /**
     * @dev 检索存储的数值
     * @return 当前存储的数值
     */
    function retrieve() public view returns (uint256) {
        return _value;
    }

    /**
     * @dev 获取合约版本
     * @return 版本字符串
     */
    function version() public pure returns (string memory) {
        return "1.0";
    }
}
```

5.2.2 BoxV2.sol - 升级版本

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./BoxV1.sol";

/**
 * @title BoxV2
 * @dev 透明代理升级演示合约 - 第二版
 * 在V1基础上增加了递增功能和历史记录
 */
contract BoxV2 is BoxV1 {
    // 新增：操作历史记录
    uint256[] private _history;

    // 新增：递增计数器
    uint256 private _incrementCounter;

    // 新增事件
    event Incremented(uint256 newValue);
    event HistoryCleared();

    /**
     * @dev 递增存储的数值
     * @return 递增后的数值
     */
    function increment() public returns (uint256) {
        uint256 newValue = retrieve() + 1;
        store(newValue);

        // 记录历史
        _history.push(newValue);
        _incrementCounter++;
```

```
        emit Incremented(newValue);
        return newValue;
    }

    /**
     * @dev 批量递增
     * @param times 递增次数
     */
    function incrementBatch(uint256 times) public {
        require(times > 0 && times <= 100, "Invalid increment times");

        for (uint256 i = 0; i < times; i++) {
            increment();
        }
    }

    /**
     * @dev 获取操作历史
     * @return 历史数值数组
     */
    function getHistory() public view returns (uint256[] memory) {
        return _history;
    }

    /**
     * @dev 获取递增操作次数
     * @return 递增次数
     */
    function getIncrementCount() public view returns (uint256) {
        return _incrementCounter;
    }

    /**
     * @dev 清空历史记录（只有owner可以调用）
     */
    function clearHistory() public onlyOwner {
        delete _history;
        _incrementCounter = 0;
        emit HistoryCleared();
    }

    /**
     * @dev 重写版本函数
     * @return 版本字符串
     */
    function version() public pure override returns (string memory) {
        return "2.0";
    }

    /**
     * @dev 新增：获取详细信息
```

```
     * @return value 当前值
     * @return historyLength 历史记录长度
     * @return incrementCount 递增次数
     * @return contractVersion 合约版本
     */
    function getDetailedInfo() public view returns (
        uint256 value,
        uint256 historyLength,
        uint256 incrementCount,
        string memory contractVersion
    ) {
        return (
            retrieve(),
            _history.length,
            _incrementCounter,
            version()
        );
    }
}
```

5.3 UUPS升级Demo实现

5.3.1 BoxUUPSV1.sol - 初始版本

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";

/**
 * @title BoxUUPSV1
 * @dev UUPS升级演示合约 - 第一版
 * 实现基础的数值存储功能和UUPS升级机制
 */
contract BoxUUPSV1 is Initializable, UUPSUpgradeable, OwnableUpgradeable {
    // 存储的数值
    uint256 private _value;

    // 合约创建时间
    uint256 private _createdAt;

    // 事件定义
    event ValueChanged(uint256 indexed oldValue, uint256 indexed newValue, address indexed
changer);
    event ContractUpgraded(address indexed oldImplementation, address indexed
newImplementation);

    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() {
```

```
        _disableInitializers();
    }

    /**
     * @dev 初始化函数
     */
    function initialize() public initializer {
        __Ownable_init();
        __UUPSUpgradeable_init();
        _value = 0;
        _createdAt = block.timestamp;
    }

    /**
     * @dev 存储数值
     * @param newValue 要存储的新数值
     */
    function store(uint256 newValue) public {
        uint256 oldValue = _value;
        _value = newValue;
        emit ValueChanged(oldValue, newValue, msg.sender);
    }

    /**
     * @dev 检索存储的数值
     * @return 当前存储的数值
     */
    function retrieve() public view returns (uint256) {
        return _value;
    }

    /**
     * @dev 获取合约创建时间
     * @return 创建时间戳
     */
    function getCreatedAt() public view returns (uint256) {
        return _createdAt;
    }

    /**
     * @dev 获取合约版本
     * @return 版本字符串
     */
    function version() public pure returns (string memory) {
        return "1.0";
    }

    /**
     * @dev UUPS升级权限控制
     * 只有合约owner可以升级
     */
    function _authorizeUpgrade(address newImplementation) internal override onlyOwner {
```

```
        emit ContractUpgraded(_getImplementation(), newImplementation);
    }

    /**
     * @dev 获取当前实现合约地址
     * @return 实现合约地址
     */
    function getImplementation() public view returns (address) {
        return _getImplementation();
    }
}
```

5.3.2 BoxUUPSV2.sol - 升级版本

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./BoxUUPSV1.sol";

/**
 * @title BoxUUPSV2
 * @dev UUPS升级演示合约 - 第二版
 * 在V1基础上增加了数学运算和批处理功能
 */
contract BoxUUPSV2 is BoxUUPSV1 {
    // 新增：运算历史
    struct Operation {
        string operationType;  // 操作类型：add, subtract, multiply, divide
        uint256 operand;       // 操作数
        uint256 result;        // 结果
        uint256 timestamp;     // 时间戳
    }

    // 操作历史数组
    Operation[] private _operations;

    // 新增：统计信息
    uint256 private _totalOperations;
    mapping(string => uint256) private _operationCounts;

    // 新增事件
    event OperationPerformed(string indexed operationType, uint256 operand, uint256
result);
    event BatchOperationCompleted(uint256 operationsCount);

    /**
     * @dev 加法操作
     * @param operand 要加的数
     * @return 运算结果
     */
    function add(uint256 operand) public returns (uint256) {
```

```
        uint256 oldValue = retrieve();
        uint256 newValue = oldValue + operand;
        store(newValue);

        _recordOperation("add", operand, newValue);
        return newValue;
    }

    /**
     * @dev 减法操作
     * @param operand 要减的数
     * @return 运算结果
     */
    function subtract(uint256 operand) public returns (uint256) {
        uint256 oldValue = retrieve();
        require(oldValue >= operand, "Result would be negative");

        uint256 newValue = oldValue - operand;
        store(newValue);

        _recordOperation("subtract", operand, newValue);
        return newValue;
    }

    /**
     * @dev 乘法操作
     * @param operand 要乘的数
     * @return 运算结果
     */
    function multiply(uint256 operand) public returns (uint256) {
        uint256 oldValue = retrieve();
        uint256 newValue = oldValue * operand;
        store(newValue);

        _recordOperation("multiply", operand, newValue);
        return newValue;
    }

    /**
     * @dev 除法操作
     * @param operand 要除的数
     * @return 运算结果
     */
    function divide(uint256 operand) public returns (uint256) {
        require(operand != 0, "Division by zero");

        uint256 oldValue = retrieve();
        uint256 newValue = oldValue / operand;
        store(newValue);

        _recordOperation("divide", operand, newValue);
        return newValue;
```

```
    }

    /**
     * @dev 批量操作
     * @param operations 操作数组：[操作类型, 操作数, 操作类型, 操作数, ...]
     */
    function batchOperations(uint256[] memory operations) public {
        require(operations.length % 2 == 0, "Invalid operations array");
        require(operations.length <= 20, "Too many operations");

        uint256 operationsCount = operations.length / 2;

        for (uint256 i = 0; i < operationsCount; i++) {
            uint256 opType = operations[i * 2];
            uint256 operand = operations[i * 2 + 1];

            if (opType == 1) {
                add(operand);
            } else if (opType == 2) {
                subtract(operand);
            } else if (opType == 3) {
                multiply(operand);
            } else if (opType == 4) {
                divide(operand);
            }
        }

        emit BatchOperationCompleted(operationsCount);
    }

    /**
     * @dev 获取操作历史
     * @param limit 返回的最大记录数
     * @return 操作历史数组
     */
    function getOperationHistory(uint256 limit) public view returns (Operation[] memory) {
        uint256 length = _operations.length;
        if (limit > length) {
            limit = length;
        }

        Operation[] memory result = new Operation[](limit);
        for (uint256 i = 0; i < limit; i++) {
            result[i] = _operations[length - 1 - i]; // 倒序返回，最新的在前
        }

        return result;
    }

    /**
     * @dev 获取统计信息
     * @return totalOps 总操作数
```

```
     * @return addCount 加法次数
     * @return subCount 减法次数
     * @return mulCount 乘法次数
     * @return divCount 除法次数
     */
    function getStatistics() public view returns (
        uint256 totalOps,
        uint256 addCount,
        uint256 subCount,
        uint256 mulCount,
        uint256 divCount
    ) {
        return (
            _totalOperations,
            _operationCounts["add"],
            _operationCounts["subtract"],
            _operationCounts["multiply"],
            _operationCounts["divide"]
        );
    }

    /**
     * @dev 清空操作历史（只有owner可以调用）
     */
    function clearHistory() public onlyOwner {
        delete _operations;
        _totalOperations = 0;

        // 清空统计
        _operationCounts["add"] = 0;
        _operationCounts["subtract"] = 0;
        _operationCounts["multiply"] = 0;
        _operationCounts["divide"] = 0;
    }

    /**
     * @dev 重写版本函数
     */
    function version() public pure override returns (string memory) {
        return "2.0";
    }

    /**
     * @dev 记录操作历史的内部函数
     */
    function _recordOperation(string memory operationType, uint256 operand, uint256
result) private {
        _operations.push(Operation({
            operationType: operationType,
            operand: operand,
            result: result,
            timestamp: block.timestamp
```

```
        }));

        _totalOperations++;
        _operationCounts[operationType]++;

        emit OperationPerformed(operationType, operand, result);
    }
}
```

5.4 Hardhat配置和依赖

5.4.1 package.json

```
{
  "name": "upgradeable-contracts-demo",
  "version": "1.0.0",
  "description": "Solidity合约升级演示项目",
  "scripts": {
    "compile": "npx hardhat compile",
    "test": "npx hardhat test",
    "deploy:transparent:local": "npx hardhat run scripts/deploy-transparent.js --network
localhost",
    "deploy:uups:local": "npx hardhat run scripts/deploy-uups.js --network localhost",
    "upgrade:transparent:local": "npx hardhat run scripts/upgrade-transparent.js --network
localhost",
    "upgrade:uups:local": "npx hardhat run scripts/upgrade-uups.js --network localhost",
    "node": "npx hardhat node"
  },
  "devDependencies": {
    "@nomicfoundation/hardhat-toolbox": "^3.0.0",
    "@openzeppelin/hardhat-upgrades": "^1.28.0",
    "hardhat": "^2.17.0"
  },
  "dependencies": {
    "@openzeppelin/contracts": "^4.9.0",
    "@openzeppelin/contracts-upgradeable": "^4.9.0"
  }
}
```

5.4.2 hardhat.config.js

```
require("@nomicfoundation/hardhat-toolbox");
require("@openzeppelin/hardhat-upgrades");

/** @type import('hardhat/config').HardhatUserConfig */
module.exports = {
  solidity: {
    version: "0.8.19",
    settings: {
      optimizer: {
```

```
        enabled: true,
        runs: 200
      }
    }
  },
  networks: {
    localhost: {
      url: "http://127.0.0.1:8545"
    },
    hardhat: {
      chainId: 1337
    }
  },
  paths: {
    sources: "./contracts",
    tests: "./test",
    cache: "./cache",
    artifacts: "./artifacts"
  }
};
```

6. 部署和测试指南

6.1 透明代理部署脚本

6.1.1 deploy-transparent.js

```
const { ethers, upgrades } = require("hardhat");

async function main() {
  console.log("开始部署透明代理合约...");

  // 获取部署账户
  const [deployer] = await ethers.getSigners();
  console.log("部署账户:", deployer.address);
  console.log("账户余额:", ethers.utils.formatEther(await deployer.getBalance()));

  // 获取合约工厂
  const BoxV1 = await ethers.getContractFactory("BoxV1");

  console.log("部署透明代理合约中...");

  // 部署透明代理合约
  const box = await upgrades.deployProxy(
    BoxV1,
    [], // 初始化参数
    {
      initializer: 'initialize',
      kind: 'transparent' // 指定使用透明代理
    }
```

```
  );

  await box.deployed();

  console.log("合约部署成功!");
  console.log("代理合约地址:", box.address);

  // 获取实现合约地址
  const implementationAddress = await
upgrades.erc1967.getImplementationAddress(box.address);
  const adminAddress = await upgrades.erc1967.getAdminAddress(box.address);

  console.log("实现合约地址:", implementationAddress);
  console.log("管理员地址:", adminAddress);

  // 测试基本功能
  console.log("\n测试基本功能...");

  // 测试初始值
  let value = await box.retrieve();
  console.log("初始值:", value.toString());

  // 测试存储功能
  console.log("存储数值 42...");
  const tx = await box.store(42);
  await tx.wait();

  value = await box.retrieve();
  console.log("存储后的值:", value.toString());

  // 获取版本信息
  const version = await box.version();
  console.log("合约版本:", version);

  // 保存部署信息
  const deploymentInfo = {
    network: "localhost",
    proxyAddress: box.address,
    implementationAddress: implementationAddress,
    adminAddress: adminAddress,
    version: version,
    deployedAt: new Date().toISOString()
  };

  console.log("\n部署信息:");
  console.log(JSON.stringify(deploymentInfo, null, 2));

  // 将部署信息保存到文件
  const fs = require('fs');
  fs.writeFileSync('transparent-deployment.json', JSON.stringify(deploymentInfo, null,
2));
  console.log("部署信息已保存到 transparent-deployment.json");
```

```
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error("部署失败:", error);
    process.exit(1);
  });
```

6.1.2 upgrade-transparent.js

```
const { ethers, upgrades } = require("hardhat");

async function main() {
  console.log("开始升级透明代理合约...");

  // 读取部署信息
  const fs = require('fs');
  let deploymentInfo;
  try {
    deploymentInfo = JSON.parse(fs.readFileSync('transparent-deployment.json', 'utf8'));
  } catch (error) {
    console.error("无法读取部署信息，请先部署合约");
    process.exit(1);
  }

  const proxyAddress = deploymentInfo.proxyAddress;
  console.log("代理合约地址:", proxyAddress);

  // 获取部署账户
  const [deployer] = await ethers.getSigners();
  console.log("升级账户:", deployer.address);

  // 连接到现有的代理合约
  const BoxV1 = await ethers.getContractFactory("BoxV1");
  const existingBox = BoxV1.attach(proxyAddress);

  // 测试升级前的状态
  console.log("\n升级前测试...");
  let value = await existingBox.retrieve();
  let version = await existingBox.version();
  console.log("当前值:", value.toString());
  console.log("当前版本:", version);

  // 存储一些测试数据
  console.log("存储测试数据...");
  await existingBox.store(100);
  value = await existingBox.retrieve();
  console.log("存储的测试值:", value.toString());

  // 获取新的合约工厂
```

```
  const BoxV2 = await ethers.getContractFactory("BoxV2");

  console.log("执行升级...");

  // 升级合约
  const upgradedBox = await upgrades.upgradeProxy(proxyAddress, BoxV2);

  console.log("升级完成!");

  // 测试升级后的功能
  console.log("\n测试升级后的功能...");

  // 检查数据是否保持
  value = await upgradedBox.retrieve();
  version = await upgradedBox.version();
  console.log("升级后的值:", value.toString());
  console.log("升级后的版本:", version);

  // 测试新功能
  console.log("测试新功能 - increment()...");
  const incrementTx = await upgradedBox.increment();
  await incrementTx.wait();

  value = await upgradedBox.retrieve();
  console.log("increment后的值:", value.toString());

  // 测试批量递增
  console.log("测试批量递增功能...");
  const batchTx = await upgradedBox.incrementBatch(5);
  await batchTx.wait();

  value = await upgradedBox.retrieve();
  const incrementCount = await upgradedBox.getIncrementCount();
  console.log("批量递增后的值:", value.toString());
  console.log("总递增次数:", incrementCount.toString());

  // 测试历史记录
  const history = await upgradedBox.getHistory();
  console.log("操作历史长度:", history.length);
  console.log("最近5次操作:", history.slice(-5).map(h => h.toString()));

  // 测试详细信息
  const detailedInfo = await upgradedBox.getDetailedInfo();
  console.log("\n详细信息:");
  console.log("当前值:", detailedInfo.value.toString());
  console.log("历史记录长度:", detailedInfo.historyLength.toString());
  console.log("递增次数:", detailedInfo.incrementCount.toString());
  console.log("合约版本:", detailedInfo.contractVersion);

  // 更新部署信息
  const newImplementationAddress = await
upgrades.erc1967.getImplementationAddress(proxyAddress);
```

```
  deploymentInfo.implementationAddress = newImplementationAddress;
  deploymentInfo.version = version;
  deploymentInfo.upgradedAt = new Date().toISOString();

  fs.writeFileSync('transparent-deployment.json', JSON.stringify(deploymentInfo, null,
2));
  console.log("升级信息已更新到 transparent-deployment.json");

  console.log("\n透明代理升级测试完成!");
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error("升级失败:", error);
    process.exit(1);
  });
```

6.2 UUPS升级部署脚本

6.2.1 deploy-uups.js

```
const { ethers, upgrades } = require("hardhat");

async function main() {
  console.log("开始部署UUPS代理合约...");

  // 获取部署账户
  const [deployer] = await ethers.getSigners();
  console.log("部署账户:", deployer.address);
  console.log("账户余额:", ethers.utils.formatEther(await deployer.getBalance()));

  // 获取合约工厂
  const BoxUUPSV1 = await ethers.getContractFactory("BoxUUPSV1");

  console.log("部署UUPS代理合约中...");

  // 部署UUPS代理合约
  const box = await upgrades.deployProxy(
    BoxUUPSV1,
    [], // 初始化参数
    {
      initializer: 'initialize',
      kind: 'uups' // 指定使用UUPS代理
    }
  );

  await box.deployed();

  console.log("合约部署成功!");
  console.log("代理合约地址:", box.address);
```

```
  // 获取实现合约地址
  const implementationAddress = await box.getImplementation();

  console.log("实现合约地址:", implementationAddress);

  // 测试基本功能
  console.log("\n测试基本功能...");

  // 测试初始值
  let value = await box.retrieve();
  console.log("初始值:", value.toString());

  // 测试存储功能
  console.log("存储数值 888...");
  const tx = await box.store(888);
  await tx.wait();

  value = await box.retrieve();
  console.log("存储后的值:", value.toString());

  // 获取版本和创建时间
  const version = await box.version();
  const createdAt = await box.getCreatedAt();
  console.log("合约版本:", version);
  console.log("创建时间:", new Date(createdAt.toNumber() * 1000).toLocaleString());

  // 保存部署信息
  const deploymentInfo = {
    network: "localhost",
    proxyAddress: box.address,
    implementationAddress: implementationAddress,
    version: version,
    createdAt: createdAt.toString(),
    deployedAt: new Date().toISOString()
  };

  console.log("\n部署信息:");
  console.log(JSON.stringify(deploymentInfo, null, 2));

  // 将部署信息保存到文件
  const fs = require('fs');
  fs.writeFileSync('uups-deployment.json', JSON.stringify(deploymentInfo, null, 2));
  console.log("部署信息已保存到 uups-deployment.json");
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error("部署失败:", error);
    process.exit(1);
  });
```

6.2.2 upgrade-uups.js

```
const { ethers, upgrades } = require("hardhat");

async function main() {
  console.log("开始升级UUPS代理合约...");

  // 读取部署信息
  const fs = require('fs');
  let deploymentInfo;
  try {
    deploymentInfo = JSON.parse(fs.readFileSync('uups-deployment.json', 'utf8'));
  } catch (error) {
    console.error("无法读取部署信息，请先部署合约");
    process.exit(1);
  }

  const proxyAddress = deploymentInfo.proxyAddress;
  console.log("代理合约地址:", proxyAddress);

  // 获取部署账户
  const [deployer] = await ethers.getSigners();
  console.log("升级账户:", deployer.address);

  // 连接到现有的代理合约
  const BoxUUPSV1 = await ethers.getContractFactory("BoxUUPSV1");
  const existingBox = BoxUUPSV1.attach(proxyAddress);

  // 测试升级前的状态
  console.log("\n升级前测试...");
  let value = await existingBox.retrieve();
  let version = await existingBox.version();
  const createdAt = await existingBox.getCreatedAt();
  console.log("当前值:", value.toString());
  console.log("当前版本:", version);
  console.log("创建时间:", new Date(createdAt.toNumber() * 1000).toLocaleString());

  // 存储一些测试数据
  console.log("存储测试数据...");
  await existingBox.store(500);
  value = await existingBox.retrieve();
  console.log("存储的测试值:", value.toString());

  // 获取新的合约工厂
  const BoxUUPSV2 = await ethers.getContractFactory("BoxUUPSV2");

  console.log("执行升级...");

  // 升级合约
  const upgradedBox = await upgrades.upgradeProxy(proxyAddress, BoxUUPSV2);

  console.log("升级完成!");
```

```
// 测试升级后的功能
console.log("\n测试升级后的功能...");

// 检查数据是否保持
value = await upgradedBox.retrieve();
version = await upgradedBox.version();
console.log("升级后的值:", value.toString());
console.log("升级后的版本:", version);

// 测试新的数学运算功能
console.log("\n测试数学运算功能...");

// 加法
console.log("执行加法: 500 + 100");
await upgradedBox.add(100);
value = await upgradedBox.retrieve();
console.log("加法后的值:", value.toString());

// 乘法
console.log("执行乘法: 600 * 2");
await upgradedBox.multiply(2);
value = await upgradedBox.retrieve();
console.log("乘法后的值:", value.toString());

// 减法
console.log("执行减法: 1200 - 200");
await upgradedBox.subtract(200);
value = await upgradedBox.retrieve();
console.log("减法后的值:", value.toString());

// 除法
console.log("执行除法: 1000 / 10");
await upgradedBox.divide(10);
value = await upgradedBox.retrieve();
console.log("除法后的值:", value.toString());

// 测试批量操作
console.log("\n测试批量操作...");
// 操作数组格式: [操作类型, 操作数, 操作类型, 操作数, ...]
// 操作类型: 1=加法, 2=减法, 3=乘法, 4=除法
const operations = [
  1, 50,  // 加50
  3, 2,   // 乘2
  2, 100, // 减100
  4, 2    // 除2
];

console.log("执行批量操作: +50, *2, -100, /2");
await upgradedBox.batchOperations(operations);
value = await upgradedBox.retrieve();
console.log("批量操作后的值:", value.toString());
```

```
  // 获取统计信息
  console.log("\n获取统计信息...");
  const stats = await upgradedBox.getStatistics();
  console.log("总操作数:", stats.totalOps.toString());
  console.log("加法次数:", stats.addCount.toString());
  console.log("减法次数:", stats.subCount.toString());
  console.log("乘法次数:", stats.mulCount.toString());
  console.log("除法次数:", stats.divCount.toString());

  // 获取操作历史
  console.log("\n获取操作历史 (最近5条)...");
  const history = await upgradedBox.getOperationHistory(5);
  console.log("历史记录:");
  for (let i = 0; i < history.length; i++) {
    const op = history[i];
    const timestamp = new Date(op.timestamp.toNumber() * 1000).toLocaleTimeString();
    console.log(`${i + 1}. ${op.operationType} ${op.operand.toString()} =
${op.result.toString()} (${timestamp})`);
  }

  // 更新部署信息
  const newImplementationAddress = await upgradedBox.getImplementation();
  deploymentInfo.implementationAddress = newImplementationAddress;
  deploymentInfo.version = version;
  deploymentInfo.upgradedAt = new Date().toISOString();

  fs.writeFileSync('uups-deployment.json', JSON.stringify(deploymentInfo, null, 2));
  console.log("升级信息已更新到 uups-deployment.json");

  console.log("\nUUPS代理升级测试完成!");
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error("升级失败:", error);
    process.exit(1);
  });
```

6.3 综合测试脚本

6.3.1 test/comprehensive-test.js

```
const { expect } = require("chai");
const { ethers, upgrades } = require("hardhat");

describe("合约升级综合测试", function () {
  let deployer, user1, user2;
  let transparentBox, uupsBox;
  let transparentBoxV2, uupsBoxV2;
```

```
before(async function () {
  [deployer, user1, user2] = await ethers.getSigners();
  console.log(`测试账户: ${deployer.address}, ${user1.address}, ${user2.address}`);
});

describe("透明代理升级测试", function () {
  it("应该成功部署透明代理", async function () {
    const BoxV1 = await ethers.getContractFactory("BoxV1");

    transparentBox = await upgrades.deployProxy(
      BoxV1,
      [],
      { initializer: 'initialize', kind: 'transparent' }
    );

    await transparentBox.deployed();
    expect(await transparentBox.version()).to.equal("1.0");
    expect(await transparentBox.retrieve()).to.equal(0);
  });

  it("应该正确存储和检索数据", async function () {
    await transparentBox.store(123);
    expect(await transparentBox.retrieve()).to.equal(123);
  });

  it("应该成功升级到V2", async function () {
    const BoxV2 = await ethers.getContractFactory("BoxV2");

    transparentBoxV2 = await upgrades.upgradeProxy(transparentBox.address, BoxV2);

    // 数据应该保持
    expect(await transparentBoxV2.retrieve()).to.equal(123);
    expect(await transparentBoxV2.version()).to.equal("2.0");
  });

  it("应该支持新功能", async function () {
    // 测试increment功能
    await transparentBoxV2.increment();
    expect(await transparentBoxV2.retrieve()).to.equal(124);

    // 测试批量increment
    await transparentBoxV2.incrementBatch(5);
    expect(await transparentBoxV2.retrieve()).to.equal(129);
    expect(await transparentBoxV2.getIncrementCount()).to.equal(6);
  });

  it("应该正确记录历史", async function () {
    const history = await transparentBoxV2.getHistory();
    expect(history.length).to.equal(6);
    expect(history[history.length - 1]).to.equal(129);
  });
```

```
  it("只有owner可以清空历史", async function () {
    // 非owner调用应该失败
    await expect(
      transparentBoxV2.connect(user1).clearHistory()
    ).to.be.revertedWith("Ownable: caller is not the owner");

    // owner调用应该成功
    await transparentBoxV2.clearHistory();
    expect(await transparentBoxV2.getIncrementCount()).to.equal(0);
  });
});

describe("UUPS升级测试", function () {
  it("应该成功部署UUPS代理", async function () {
    const BoxUUPSV1 = await ethers.getContractFactory("BoxUUPSV1");

    uupsBox = await upgrades.deployProxy(
      BoxUUPSV1,
      [],
      { initializer: 'initialize', kind: 'uups' }
    );

    await uupsBox.deployed();
    expect(await uupsBox.version()).to.equal("1.0");
    expect(await uupsBox.retrieve()).to.equal(0);
  });

  it("应该正确存储和检索数据", async function () {
    await uupsBox.store(456);
    expect(await uupsBox.retrieve()).to.equal(456);
  });

  it("应该成功升级到V2", async function () {
    const BoxUUPSV2 = await ethers.getContractFactory("BoxUUPSV2");

    uupsBoxV2 = await upgrades.upgradeProxy(uupsBox.address, BoxUUPSV2);

    // 数据应该保持
    expect(await uupsBoxV2.retrieve()).to.equal(456);
    expect(await uupsBoxV2.version()).to.equal("2.0");
  });

  it("应该支持数学运算", async function () {
    // 测试加法
    await uupsBoxV2.add(44);
    expect(await uupsBoxV2.retrieve()).to.equal(500);

    // 测试乘法
    await uupsBoxV2.multiply(2);
    expect(await uupsBoxV2.retrieve()).to.equal(1000);
```

```
      // 测试减法
      await uupsBoxV2.subtract(200);
      expect(await uupsBoxV2.retrieve()).to.equal(800);

      // 测试除法
      await uupsBoxV2.divide(4);
      expect(await uupsBoxV2.retrieve()).to.equal(200);
    });

    it("应该支持批量操作", async function () {
      // 批量操作: +100, *3, -50, /2
      const operations = [1, 100, 3, 3, 2, 50, 4, 2];
      await uupsBoxV2.batchOperations(operations);

      // (200 + 100) * 3 - 50) / 2 = (900 - 50) / 2 = 425
      expect(await uupsBoxV2.retrieve()).to.equal(425);
    });

    it("应该正确统计操作", async function () {
      const stats = await uupsBoxV2.getStatistics();
      expect(stats.addCount).to.equal(2); // add(44) + 批量中的add(100)
      expect(stats.mulCount).to.equal(2); // multiply(2) + 批量中的multiply(3)
      expect(stats.subCount).to.equal(2); // subtract(200) + 批量中的subtract(50)
      expect(stats.divCount).to.equal(2); // divide(4) + 批量中的divide(2)
    });

    it("只有owner可以升级", async function () {
      const BoxUUPSV2 = await ethers.getContractFactory("BoxUUPSV2");
      const newImpl = await BoxUUPSV2.deploy();

      // 非owner调用应该失败
      await expect(
        uupsBoxV2.connect(user1).upgradeTo(newImpl.address)
      ).to.be.revertedWith("Ownable: caller is not the owner");
    });

    it("应该防止除零错误", async function () {
      await expect(
        uupsBoxV2.divide(0)
      ).to.be.revertedWith("Division by zero");
    });

    it("应该防止负数结果", async function () {
      // 当前值是425，减去500应该失败
      await expect(
        uupsBoxV2.subtract(500)
      ).to.be.revertedWith("Result would be negative");
    });
  });

  describe("Gas费用对比测试", function () {
    it("应该显示两种升级模式的Gas差异", async function () {
```

```
      // 透明代理的gas费用
      const transparentTx = await transparentBoxV2.retrieve();
      const transparentReceipt = await transparentTx.wait();

      // UUPS代理的gas费用
      const uupsTx = await uupsBoxV2.retrieve();
      const uupsReceipt = await uupsTx.wait();

      console.log(`透明代理调用Gas: ${transparentReceipt.gasUsed}`);
      console.log(`UUPS代理调用Gas: ${uupsReceipt.gasUsed}`);

      // UUPS应该更节省Gas
      expect(uupsReceipt.gasUsed.lt(transparentReceipt.gasUsed)).to.be.true;
    });
  });
});
```

6.4 运行测试的完整步骤

6.4.1 环境准备

```
# 1. 创建项目目录
mkdir upgradeable-contracts-demo
cd upgradeable-contracts-demo

# 2. 初始化项目
npm init -y

# 3. 安装依赖
npm install --save-dev @nomicfoundation/hardhat-toolbox @openzeppelin/hardhat-upgrades
hardhat
npm install @openzeppelin/contracts @openzeppelin/contracts-upgradeable

# 4. 初始化Hardhat
npx hardhat

# 5. 创建目录结构
mkdir -p contracts/transparent contracts/uups scripts test
```

6.4.2 运行测试步骤

```
# 1. 编译合约
npx hardhat compile

# 2. 启动本地节点
npx hardhat node

# 3. 在新终端中运行测试
npx hardhat test
```

```
# 4. 部署透明代理
npx hardhat run scripts/deploy-transparent.js --network localhost

# 5. 升级透明代理
npx hardhat run scripts/upgrade-transparent.js --network localhost

# 6. 部署UUPS代理
npx hardhat run scripts/deploy-uups.js --network localhost

# 7. 升级UUPS代理
npx hardhat run scripts/upgrade-uups.js --network localhost
```

---

7. 最佳实践和安全考虑

7.1 存储布局兼容性

关键原则：新版本合约的存储布局必须与旧版本兼容

```
// 错误的升级方式 - 改变了存储布局
contract BoxV1 {
    uint256 private _value;
    address private _owner;
}

contract BoxV2 {
    address private _owner;  // 位置改变了
    uint256 private _value;  // 位置改变了
    uint256 private _newVar; // 新变量
}

// 正确的升级方式 - 保持存储布局
contract BoxV1 {
    uint256 private _value;
    address private _owner;
}

contract BoxV2 {
    uint256 private _value;  // 位置不变
    address private _owner;  // 位置不变
    uint256 private _newVar; // 在末尾添加
}
```

7.2 初始化安全性

```
// 正确的初始化模式
contract MyUpgradeableContract is Initializable {
    uint256 private _value;

    /// @custom:oz-upgrades-unsafe-allow constructor
```

```
    constructor() {
        _disableInitializers(); // 防止实现合约被初始化
    }

    function initialize(uint256 initialValue) public initializer {
        _value = initialValue;
    }

    // 如需要再次初始化
    function reinitialize(uint256 newValue) public reinitializer(2) {
        _value = newValue;
    }
}
```

7.3 权限控制最佳实践

```
// 透明代理权限控制
contract MyTransparentContract is OwnableUpgradeable {
    function initialize() public initializer {
        __Ownable_init();
    }

    // 业务函数不需要考虑升级权限
    function businessFunction() public {
        // 业务逻辑
    }
}

// UUPS权限控制
contract MyUUPSContract is UUPSUpgradeable, OwnableUpgradeable {
    function initialize() public initializer {
        __Ownable_init();
        __UUPSUpgradeable_init();
    }

    function _authorizeUpgrade(address newImplementation)
        internal
        override
        onlyOwner
    {
        // 可以添加额外的升级条件检查
        require(newImplementation != address(0), "Invalid implementation");

        // 可以添加版本检查
        // IVersioned(newImplementation).version() > currentVersion
    }
}
```

7.4 升级时的数据迁移

```
contract BoxV3 is BoxV2 {
    // 新的存储变量
    mapping(address => uint256) private _userBalances;
    bool private _migrated;

    function migrateData() external onlyOwner {
        require(!_migrated, "Already migrated");

        // 执行数据迁移逻辑
        _userBalances[msg.sender] = retrieve();
        _migrated = true;
    }

    function version() public pure override returns (string memory) {
        return "3.0";
    }
}
```

7.6 常见陷阱和避免方法

7.6.1 构造函数陷阱

```
// 错误 - 升级合约中使用构造函数
contract BadUpgradeable {
    uint256 private _value;

    constructor(uint256 initialValue) {
        _value = initialValue; // 在代理模式中不会执行
    }
}

// 正确 - 使用初始化函数
contract GoodUpgradeable is Initializable {
    uint256 private _value;

    function initialize(uint256 initialValue) public initializer {
        _value = initialValue;
    }
}
```

7.6.2 存储冲突

```
// 危险 - 可能的存储冲突
contract ProxyContract {
    address implementation; // slot 0
    address admin;          // slot 1
}

contract LogicContract {
    uint256 value;   // slot 0 - 冲突!
```

```
    address owner;   // slot 1 - 冲突!
}

// 安全 - 使用存储插槽
contract SafeLogicContract {
    // 使用特定插槽避免冲突
    bytes32 private constant VALUE_SLOT = keccak256("my.contract.value");

    function getValue() public view returns (uint256) {
        bytes32 slot = VALUE_SLOT;
        uint256 value;
        assembly {
            value := sload(slot)
        }
        return value;
    }
}
```

7.7 监控和治理

7.7.1 升级事件监控

```
contract MonitoredUpgradeable is UUPSUpgradeable, OwnableUpgradeable {
    event UpgradeProposed(address indexed newImplementation, uint256 delay);
    event UpgradeExecuted(address indexed oldImplementation, address indexed
newImplementation);
    event UpgradeCancelled(address indexed proposedImplementation);

    struct UpgradeProposal {
        address newImplementation;
        uint256 proposalTime;
        uint256 executeTime;
        bool executed;
    }

    UpgradeProposal public pendingUpgrade;
    uint256 public constant UPGRADE_DELAY = 2 days;

    function proposeUpgrade(address newImplementation) external onlyOwner {
        require(newImplementation != address(0), "Invalid implementation");
        require(!pendingUpgrade.executed, "Upgrade already pending");

        pendingUpgrade = UpgradeProposal({
            newImplementation: newImplementation,
            proposalTime: block.timestamp,
            executeTime: block.timestamp + UPGRADE_DELAY,
            executed: false
        });

        emit UpgradeProposed(newImplementation, UPGRADE_DELAY);
    }
```

```
    function executeUpgrade() external onlyOwner {
        require(pendingUpgrade.newImplementation != address(0), "No pending upgrade");
        require(block.timestamp >= pendingUpgrade.executeTime, "Upgrade delay not met");
        require(!pendingUpgrade.executed, "Upgrade already executed");

        address oldImplementation = _getImplementation();
        pendingUpgrade.executed = true;

        _upgradeToAndCallUUPS(pendingUpgrade.newImplementation, new bytes(0), false);

        emit UpgradeExecuted(oldImplementation, pendingUpgrade.newImplementation);
    }

    function _authorizeUpgrade(address newImplementation) internal override onlyOwner {
        // 升级授权逻辑已在executeUpgrade中处理
    }
}
```

二种代理模式总结

本文档详细介绍了Solidity合约的两种主要升级模式：透明代理升级和UUPS升级。通过理论解释、完整代码示例和实战演练：

1. **理解升级原理**：掌握代理模式和delegatecall机制
2. **选择合适模式**：根据项目需求选择透明代理或UUPS
3. **安全实施升级**：避免常见陷阱，确保升级安全
4. **监控和治理**：建立完善的升级治理机制

关键要点回顾：

- **透明代理**：更安全但Gas成本较高，适合高价值协议
- **UUPS**：更高效但需要谨慎实施，适合高频应用
- **存储布局**：升级时必须保持兼容性
- **权限控制**：确保只有授权地址可以执行升级
- **测试覆盖**：充分测试各种升级场景